PolliNexus API — Manual Test Guide (mirrors pollinexus-done.ipynb)

Introduction

PolliNexus is a comprehensive data science platform that combines ecological analysis with production-grade machine learning pipelines. The system follows a full-stack data mindset: defining clear hypotheses, assessing data quality, and applying domain-aware cleaning before modeling.

The platform favors interpretable methods and reports metrics honestly, highlighting feature importance and limitations. On the engineering side, it implements secure, observable, and maintainable systems—validation on inputs, structured logging, health/metrics endpoints, and clear API contracts—so analysis scales from a notebook into reliable services.

System Architecture Overview

Core Design Principles

PolliNexus follows a **microservice-inspired architecture** with clear separation of concerns:

- **API Layer** (`src/pollinexus/api/`): FastAPI-based REST endpoints with automatic OpenAPI documentation
- **Core Services** (`src/pollinexus/core/`): Authentication, configuration, database, logging, metrics, and security
- **Data Services** (`src/pollinexus/services/`): Business logic for data processing, database operations, and DuckDB analytics
- **Task Processing** (`src/pollinexus/tasks/`): Celery-based asynchronous job processing for ML and visualization
- **Models** (`src/pollinexus/models/`): Pydantic schemas for request/response validation and SQLAlchemy ORM models

Data Science Pipeline Architecture

The system implements a **production-grade ML pipeline**:

1. **Data Ingestion**: CSV upload with automatic schema detection and validation
2. **Data Quality Assessment**: Missing value analysis, type validation, domain-specific checks
3. **Feature Engineering**: Temporal features (month, season), categorical encoding, composite metrics
4. **Model Training**: Interpretable ML (Random Forest) with feature importance analysis
5. **Result Generation**: Structured outputs with confidence scores and business metrics
6. **Visualization**: Automated chart generation with configurable parameters

Security & Observability

- **Authentication**: JWT-based with OTP verification for secure access
- **Input Validation**: Pydantic models with comprehensive type checking
- **Structured Logging**: JSON-formatted logs with correlation IDs for traceability

- **Health Monitoring**: Comprehensive health checks with dependency status
- **Metrics Collection**: Prometheus-compatible metrics for operational insights

Follow these steps to validate the API end-to-end. Replace placeholders like , , , <DATASET_ID>, <JOB_ID>, <VIS_ID>, <TASK_ID>.

Reference by Tag Group

- Health & Information: docs/api-groups/api-health-info.md
- User Authentication & Accounts: docs/api-groups/api-auth.md
- Dataset Management: docs/api-groups/api-datasets.md
- Data Analysis: docs/api-groups/api-analysis.md
- Visualizations: docs/api-groups/api-visualizations.md
- Monitoring: docs/api-groups/api-monitoring.md

0) Base setup (Makefile-driven)

The API can be started using the project's Makefile to ensure reproducible and aligned development workflows. This approach provides consistent development environments across different setups.

Environment Management Strategy

The Makefile provides **consistent deployment patterns** across different environments:

- **Local Development**: Uses uv for fast dependency management and virtual environment isolation
- **Docker Containerization**: Ensures reproducible deployments with all dependencies bundled
- **Remote VM**: Supports cloud deployment with proper networking and service discovery

Infrastructure Components

- **Database**: SQLite for development, PostgreSQL for production (configurable)
- **Message Queue**: Redis for Celery task processing and caching
- **File Storage**: Local filesystem with configurable upload directories
- **Monitoring**: Prometheus metrics endpoint and structured logging

- Local API (uv):

```
make run-local
```

- Docker API:

```
make run-docker
```

- Remote VM API:

```
make run-remote
```

1) Health, info, quick checks

Before any workflow, verify health, detailed status, and basic API info to ensure a clean baseline. This systematic approach validates system readiness and ensures all components are functioning properly.

Health Check Architecture

The health system implements **layered monitoring**:

- **Basic Health**: Simple alive/dead status for load balancers
- **Detailed Health**: Deep dependency checks (database, Redis, file system)
- **Metrics**: Performance counters and business KPIs
- **Info**: System configuration and version details

Monitoring Strategy

- **Dependency Health**: Database connectivity, Redis availability, disk space
- **Performance Metrics**: Request latency, error rates, queue depths
- **Business Metrics**: Dataset counts, job success rates, user activity
- **Security Status**: Authentication failures, rate limiting, suspicious activity
- Makefile health targets (local):

```
make health
make health-detailed
```

- Direct API checks:

```
curl "http://localhost:8000/api/v1/health"
curl "http://localhost:8000/api/v1/health/detailed"
curl "http://localhost:8000/api/v1/info"
curl "http://localhost:8000/api/v1/metrics"
```

2) (Optional) Auth flow to get a JWT

To demonstrate authentication, register, verify OTP, and fetch a JWT, then call a protected endpoint. This implements secure authentication practices with multi-factor verification.

Authentication Architecture

The auth system implements **multi-factor security**:

- **Registration Flow**: Email verification with OTP to prevent fake accounts
- **Login Flow**: OTP-based authentication for enhanced security
- **JWT Tokens**: Stateless authentication with configurable expiration
- **Rate Limiting**: Prevents brute force attacks on auth endpoints

Security Features

- **Passwordless Authentication**: Eliminates password storage and related vulnerabilities
- **OTP Expiration**: Time-limited codes prevent replay attacks
- **JWT Refresh**: Automatic token renewal with secure rotation
- **Audit Logging**: All authentication events logged for security monitoring

- Register (sends OTP to email)

```
curl -X POST "http://localhost:8000/api/v1/user/register" \
  -H "Content-Type: application/json" \
  -d '{"full_name":"Test User","email":"<EMAIL>","phone":"+10000000000"}'
```

- Send verification code again if needed

```
curl -X POST "http://localhost:8000/api/v1/user/send-verification" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>","verification_type":"email"}'
```

- Verify registration (use the OTP you received); receives JWT

```
curl -X POST "http://localhost:8000/api/v1/user/verify-registration" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>","otp":"<OTP>"}'
```

- Login OTP (later, when logging in)

```
curl -X POST "http://localhost:8000/api/v1/user/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>"}'
```

- Verify login to get JWT

```
curl -X POST "http://localhost:8000/api/v1/user/verify-login" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>","otp":"<OTP>"}'
```

- Check profile

```
curl -H "Authorization: Bearer <JWT>" "http://localhost:8000/api/v1/user/me"
```

3) Upload dataset (plants_and_bees.csv)

Upload the plants-and-bees dataset to enable downstream analysis and visualization. This applies robust data ingestion techniques with validation and quality checks.

Data Ingestion Pipeline

The upload system implements **robust data handling**:

- **File Validation**: Size limits, format checking, virus scanning
- **Schema Detection**: Automatic column type inference and validation
- **Data Quality Checks**: Missing value analysis, outlier detection
- **Metadata Extraction**: File statistics, column summaries, data lineage

Ecological Data Context

The plants_and_bees.csv contains **pollinator ecology data**:

- **Temporal Data**: Date/time of observations for seasonal analysis
- **Spatial Data**: Site locations for geographic patterns
- **Species Data**: Plant and bee species for biodiversity analysis
- **Abundance Data**: Visit counts and population estimates
- **Environmental Data**: Weather conditions, sampling methods

```
curl -X POST "http://localhost:8000/api/v1/datasets/" \
  -F "name=Plants and Bees Dataset" \
  -F "description=Sample pollinator data" \
  -F "file=@dataset/plants_and_bees.csv"
```

4) Inspect datasets

Confirm ingestion with list/get operations, then pull dataset info and a health check to validate integrity and statistics. This follows comprehensive data validation practices to ensure data quality.

Dataset Management Architecture

The dataset system provides **comprehensive data governance**:

- **CRUD Operations**: Create, read, update, delete with proper validation
- **Metadata Management**: Rich descriptions, tags, versioning
- **Data Profiling**: Automatic statistics generation and quality metrics

- **Access Control**: User-based permissions and audit trails

Data Quality Framework

- **Completeness**: Missing value analysis and imputation strategies
- **Consistency**: Data type validation and range checking
- **Accuracy**: Domain-specific validation rules
- **Timeliness**: Data freshness and update frequency tracking

```
# List
curl "http://localhost:8000/api/v1/datasets/?skip=0&limit=20"
# Get one by id
curl "http://localhost:8000/api/v1/datasets/<DATASET_ID>"
# Dataset info
curl "http://localhost:8000/api/v1/datasets/<DATASET_ID>/info"
# Dataset health
curl "http://localhost:8000/api/v1/datasets/<DATASET_ID>/health"
```

5) Start ML analysis

With data in place, trigger ML workflows (bee preferences, recommendations) to generate analytical insights. This implements production-ready machine learning pipelines with interpretable models and comprehensive validation.

Machine Learning Architecture

The ML system implements **production-ready analytics**:

- **Feature Engineering**: Domain-specific transformations for ecological data
- **Model Selection**: Interpretable algorithms (Random Forest, Logistic Regression)
- **Validation Strategy**: Stratified sampling, cross-validation, holdout sets
- **Result Interpretation**: Feature importance, confidence intervals, business metrics

Ecological Analysis Methods

- **Bee Preference Modeling**: Predicts native vs non-native bee preferences
- **Plant Recommendation**: Multi-criteria ranking for conservation planning
- **Seasonal Analysis**: Temporal patterns and phenology modeling
- **Site Comparison**: Geographic variation and habitat quality assessment
- Bee preference model

```
curl -X POST "http://localhost:8000/api/v1/analysis/bee-preferences/" \
  -H "Content-Type: application/json" \
  -d '{"dataset_id": <DATASET_ID>, "target_column": "nonnative_bee", "model_type":
"random_forest", "test_size": 0.2}'
```

- (Optional) Plant recommendations

```
curl -X POST "http://localhost:8000/api/v1/analysis/plant-recommendations/" \
  -H "Content-Type: application/json" \
  -d '{"dataset_id": <DATASET_ID>, "top_n": 10, "criteria":
["bee_attraction","seasonal_availability"]}'
```

- (Optional) Seasonal/site analyses

```
curl -X POST "http://localhost:8000/api/v1/analysis/seasonal/?dataset_id=
<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"include_trends": true, "seasonal_periods": 12}'

curl -X POST "http://localhost:8000/api/v1/analysis/site-comparison/?dataset_id=
<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"metrics": ["total_bees","diversity_index"], "group_by": "site"}'
```

6) Track jobs and fetch results

Asynchronous Processing Architecture

The job system implements **scalable task processing**:

- **Celery Integration**: Distributed task queue with Redis backend
- **Job Lifecycle**: Created → Queued → Running → Completed/Failed
- **Progress Tracking**: Real-time status updates and progress indicators
- **Result Storage**: Structured outputs with metadata and artifacts

Job Management Features

- **Queue Management**: Priority queuing, retry logic, dead letter queues
- **Resource Monitoring**: CPU, memory, and disk usage tracking
- **Error Handling**: Comprehensive error logging and recovery strategies
- **Result Caching**: Temporary storage with configurable retention

```
# List jobs
curl "http://localhost:8000/api/v1/analysis/jobs/?dataset_id=<DATASET_ID>"
# Get job status
curl "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>/status"
# Get job info
curl "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>"
# Get results
curl "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>/results"
```

```
# (Optional) cancel
curl -X DELETE "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>"
```

7) Create visualizations

Visualization Architecture

The viz system provides **automated chart generation**:

- **Template System**: Pre-defined chart types with configurable parameters
- **Data Processing**: Aggregation, filtering, and transformation for visualization
- **Chart Generation**: Matplotlib/Seaborn-based with consistent styling
- **Export Options**: PNG, PDF, SVG formats with metadata

Ecological Visualization Types

- **Bee Distribution**: Species abundance and diversity patterns
- **Seasonal Patterns**: Temporal trends and phenology visualization
- **Site Comparison**: Geographic variation and habitat analysis
- **Dashboard**: Multi-panel overview for comprehensive insights

```
# Bee distribution
curl -X POST "http://localhost:8000/api/v1/visualizations/bee-distribution/?
dataset_id=<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"top_n": 20, "include_percentages": true}'
# Seasonal patterns
curl -X POST "http://localhost:8000/api/v1/visualizations/seasonal-patterns/?
dataset_id=<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"include_trends": true, "seasonal_periods": 12}'
# Site comparison
curl -X POST "http://localhost:8000/api/v1/visualizations/site-comparison/?
dataset_id=<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"metrics": ["total_bees","diversity"], "group_by": "site"}'
# Dashboard
curl -X POST "http://localhost:8000/api/v1/visualizations/dashboard/?dataset_id=
<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"include_timeline": true, "include_metrics": true}'
```

8) Track visualization tasks and download artifacts

Artifact Management

The system provides **comprehensive output handling**:

- **File Storage**: Organized directory structure with metadata

- **Access Control**: User-based permissions for artifact access
- **Version Management**: Artifact versioning and rollback capabilities
- **Export Formats**: Multiple output formats for different use cases

Download and Distribution

- **Direct Downloads**: Secure file serving with proper headers
- **Metadata Access**: Rich information about generated artifacts
- **Batch Operations**: Bulk download and export capabilities
- **Integration Ready**: API endpoints for external system integration

```
# Check task status
curl "http://localhost:8000/api/v1/visualizations/<VIS_ID>/status?task_id=
<TASK_ID>"
# Download info
curl "http://localhost:8000/api/v1/visualizations/<VIS_ID>/download?task_id=
<TASK_ID>"
# Cancel
curl -X DELETE "http://localhost:8000/api/v1/visualizations/<VIS_ID>?task_id=
<TASK_ID>"
# List available viz types
curl "http://localhost:8000/api/v1/visualizations/available-types"
```

9) Monitoring and shutdown insights

Operational Monitoring

The monitoring system provides **comprehensive observability**:

- **Application Metrics**: Request rates, response times, error rates
- **Business Metrics**: User activity, dataset usage, analysis success rates
- **System Metrics**: Resource utilization, queue depths, cache hit rates
- **Security Metrics**: Authentication attempts, rate limiting, suspicious activity

Graceful Shutdown

- **Health Degradation**: Gradual service degradation with proper signaling
- **Data Preservation**: Safe shutdown with data consistency guarantees
- **Resource Cleanup**: Proper cleanup of temporary files and connections
- **Audit Trail**: Complete shutdown logging for post-mortem analysis

```
curl "http://localhost:8000/api/v1/metrics"
curl "http://localhost:8000/api/v1/security/status"
curl "http://localhost:8000/api/v1/shutdown/status"
curl "http://localhost:8000/api/v1/shutdown/metrics"
```

Advanced Usage Patterns

Batch Processing Workflows

For production use cases, implement **automated workflows**:

```
# Automated analysis pipeline
curl -X POST "http://localhost:8000/api/v1/analysis/batch/" \
  -H "Content-Type: application/json" \
  -d '{"dataset_id": <DATASET_ID>, "analyses": ["bee-preferences", "plant-
recommendations", "seasonal"], "visualizations": ["dashboard"]}'
```

Integration Patterns

For external system integration:

```
# Webhook notifications
curl -X POST "http://localhost:8000/api/v1/webhooks/" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://your-system.com/webhook", "events": ["job.completed",
"visualization.ready"]}'
```

Notes

- Prefer Makefile targets to start services (local, docker, remote) and to check health.
- If auth is enforced, add: `-H "Authorization: Bearer <JWT>"` to protected requests.
- Use returned IDs (`dataset_id`, `job_id`, `visualization_id`, `task_id`) from earlier calls as you proceed.

Troubleshooting Guide

Common Issues and Solutions

Authentication Problems

- OTP expiration: Request new verification code
- JWT expiration: Re-authenticate to get fresh token
- Rate limiting: Wait for cooldown period

Data Processing Issues

- Large file uploads: Check file size limits and network stability
- Memory constraints: Monitor system resources during processing
- Encoding issues: Ensure CSV files use UTF-8 encoding

Job Processing Delays

- Queue backlog: Check Celery worker status and Redis connectivity
- Resource constraints: Monitor CPU and memory usage
- Network issues: Verify external service dependencies

Visualization Generation Failures

- Data quality: Ensure sufficient data for requested chart types
- Memory limits: Check available RAM for large datasets
- File permissions: Verify write access to output directories